

Sepsis Early Warning System Using Deep Learning

Complex Computing Problem Deep Learning

Santosh | 22F-BSAI-63 | Department of Artificial Intelligence | 8th Semester

Abstract

This report is about building a deep learning system that can detect sepsis early in ICU patients. The idea came from a simple but scary observation patients can be deteriorating for hours while doctors and nurses are busy elsewhere. I wanted to see if AI could help with that. I built three generations of models: first a basic DNN to get a feel for the problem, then LSTM/GRU networks because patient vitals are time-series data, and finally ClinicalBERT for reading clinical notes. The final ensemble hits around 89% accuracy and 84% recall. Recall was the metric I cared about most for sepsis, missing a real case is far worse than a false alarm.

1. The Clinical Problem

1.1 Why This Problem is Hard

Sepsis is not easy to catch. The biggest challenge is that patient vitals are not just numbers in isolation — they have to be understood in context and over time. A heart rate of 110 might be completely fine after a walk, but in an ICU patient who was sitting still and had a rate of 65 an hour ago, that same number is a red flag. The data is also really messy. Monitors disconnect, nurses chart at different times, blood draws get missed. In the dataset I used, roughly 10% of values are missing. That forced me to think carefully about imputation.

Then there's the class imbalance. Sepsis only shows up in about 2-5% of ICU admissions. If a model just says 'stable' for every patient, it gets 95% accuracy which sounds good but is completely useless. This is why I did not rely on accuracy alone and focused more on recall.

1.2 Why Text Notes Matter

At one point I asked myself why not just use the numbers? Vitals are clean-ish, structured, and easy to feed into a model. But then I thought about this situation: two patients with the exact same HR, BP, and temperature readings. For one, the nursing note says 'patient resting comfortably, ate breakfast.' For the other it says 'family reports patient has been confused since this morning, seems unresponsive to commands.' Same numbers, completely different picture. Clinical notes capture the things sensors can't what the patient looks like, how they're behaving, what the family is saying. Ignoring that would be leaving a lot of information on the table.

1.3 Recall Over Accuracy

This was an important design decision I had to justify. In sepsis prediction, a false negative means a patient possibly dies from untreated septic shock. A false positive means a nurse walks over, checks the patient, finds nothing serious, and moves on that's maybe five

minutes of time. Given that tradeoff, I would rather my model raise 100 unnecessary alarms than miss one real case. So recall (sensitivity) became the primary metric, and I tuned thresholds and class weights with that in mind.

To make this concrete: say Patient A is 67 years old, post cardiac surgery. His HR climbs to 115, temperature hits 38.8°C, systolic BP drops to 88. Lactate is at 3.2. A model optimized for accuracy might still call him stable because those values are on the edge. Missed sepsis, delayed antibiotics, dead patient within 12 hours. That's the failure mode I was trying to avoid.

2. Baseline Model DNN

2.1 Vanishing Gradients

Before building anything, I had to think about why simple deep networks fail. The vanishing gradient problem is the main one. During backprop, gradients get multiplied layer by layer. If you're using sigmoid or tanh, those derivatives are always between 0 and 1. Multiply ten of them together and you get something like 10^{-10} at the first layer basically zero. Early layers stop learning anything meaningful.

My fix was three things together: ReLU activation (derivative is 0 or 1, no decay), He initialization (scales weights by $\sqrt{2/n_{in}}$ to keep activations stable at the start), and Batch Normalization after each layer to normalize inputs and keep gradients flowing properly. Together these made a big difference.

2.2 Optimizer Choice Adam vs SGD

I ran both and compared. SGD with momentum updates weights using accumulated velocity which smooths out oscillations. Adam tracks per-parameter first and second moment estimates and adjusts learning rates individually. The update rule for Adam is:

$$\begin{aligned}m_t &= \text{beta1} * m_{(t-1)} + (1 - \text{beta1}) * g_t \\v_t &= \text{beta2} * v_{(t-1)} + (1 - \text{beta2}) * g_t^2 \\ \theta_{t+1} &= \theta_t - \text{lr} * m_{\text{hat}} / (\text{sqrt}(v_{\text{hat}}) + \text{eps})\end{aligned}$$

In practice, Adam converged in 28 epochs vs SGD's 67. But interestingly, SGD gave slightly better validation loss (0.138 vs 0.142), probably because it finds wider, more generalizable minima. I went with Adam in the end because of the speed advantage, and used early stopping to avoid overfitting.

2.3 Loss Function and Class Weighting

I used binary cross-entropy: $-[y * \log(y_{\text{hat}}) + (1-y) * \log(1 - y_{\text{hat}})]$. To handle class imbalance, I applied class weights sepsis examples were weighted 3x higher in the loss calculation. This forces the model to penalize missed sepsis cases more heavily during training.

2.4 Dropout

Dropout randomly turns off neurons during training (I used $p=0.3-0.4$). This stops neurons from co-adapting basically preventing the network from becoming overly reliant on specific activation patterns that only appear in training data. Combined with batch norm, this cut validation loss by about 12% compared to no regularization.

3. Temporal Modeling LSTM, Bi-LSTM, GRU

3.1 Why RNNs for This Problem

A DNN treats each input as independent. But patient vitals are sequential what happened an hour ago absolutely matters for interpreting what's happening now. RNNs process sequences with a hidden state that carries forward information from previous time steps. But standard RNNs have the same vanishing gradient problem, and it's actually worse because gradients now have to travel back through time as well as layers. For a 24-hour monitoring window, that's a long way to go.

3.2 LSTM Gating Mechanism

LSTM solves this with a cell state a kind of memory highway that runs straight through the sequence with minimal interference. Three gates control what gets written to or erased from it:

Gate	Formula	Purpose
Forget	$f_t = \text{sigmoid}(W_f * [h_{(t-1)}, x_t] + b_f)$	Decide what to erase from cell
Input	$i_t = \text{sigmoid}(W_i * [h_{(t-1)}, x_t] + b_i)$	Decide what new info to write
Output	$o_t = \text{sigmoid}(W_o * [h_{(t-1)}, x_t] + b_o)$	Produce the hidden state output

The key is the cell state update: $C_t = f_t * C_{(t-1)} + i_t * \tilde{C}_t$. Because this path is additive (not multiplicative across many nonlinearities), gradients can flow across hundreds of time steps without disappearing. That's how LSTM handles long-term dependencies.

3.3 GRU Simpler but Competitive

GRU merges the cell and hidden state into one and uses only two gates: an update gate (how much past info to keep) and a reset gate (how much past info to forget). It has about 25% fewer parameters than LSTM, trains faster, and is harder to overfit on smaller datasets. In my 12-hour clinical window, GRU and LSTM performed nearly identically accuracy-wise, but GRU trained 22% faster. For a system that needs frequent retraining on fresh data, that matters.

3.4 Bidirectional LSTM When to Use It and When Not To

Bi-LSTM runs two LSTMs: one forward through the sequence and one backward. Their hidden states are concatenated. This gave me 85% accuracy vs unidirectional LSTM's 82% a real improvement.

But here's the catch: Bi-LSTM is only usable for retrospective analysis, like reviewing a discharged patient's chart. In a real-time ICU dashboard, you don't have future data. At time t , $x_{(t+1)}$ through x_T don't exist yet. A bidirectional model would have to wait until discharge to generate an alert — which defeats the entire purpose of an early warning system. For live deployment, unidirectional LSTM or GRU is the only sensible choice, even at a small accuracy cost.

4. Clinical Language ClinicalBERT

4.1 Why Pre-trained Models Make Sense Here

Training a language model from scratch on 2,000 clinical notes is like handing a medical student one pamphlet and expecting them to pass their boards. ClinicalBERT was pre-trained on millions of PubMed abstracts and MIMIC-III discharge summaries. It already knows what tachycardia means, that elevated lactate signals tissue hypoperfusion, and how clinical language is structured. Fine-tuning it on my sepsis labels transfers all of that knowledge rather than relearning from scratch critical when your dataset is small because of privacy constraints.

4.2 Self-Attention vs Sequential Processing

LSTM reads a clinical note word by word. Self-attention looks at all words simultaneously and computes relevance scores between every pair. For an n-word note, this produces an n x n attention matrix. A doctor reading a note doesn't actually read linearly either they scan for key terms like 'tachycardia' or 'altered mental status' regardless of where those words appear. Self-attention replicates that behavior computationally.

4.3 What the Attention Weights Showed

After training, I visualized what terms the model was focusing on. The results were reassuring the model had basically learned clinical intuition:

Term	Attention Weight	Why It Matters Clinically
tachycardia	0.92	HR > 100 bpm; in Sepsis-3 criteria
septic	0.88	Explicit diagnostic language
shock	0.85	Hemodynamic instability, urgent
lactate	0.81	> 2 mmol/L is a sepsis criterion
elevated	0.78	Indicates abnormal lab values
hypotensive	0.75	MAP < 65 mmHg — shock state
fever	0.72	Temp > 38.3°C, infection sign
distress	0.68	Respiratory or clinical distress

This explainability is actually important for clinical adoption. Doctors won't trust a black box. If they can see that the model flagged a patient because it noticed 'tachycardia, elevated lactate, hypotensive' in the notes terms they themselves look for they're much more likely to take the alert seriously.

4.4 Frozen vs Full Fine-Tuning

I tested both approaches. Frozen fine-tuning only trains the classification head (1.2M parameters, about 1% of the model). Full fine-tuning trains all 110M parameters. Results were clear:

	Frozen Base	Full Fine-Tuning
Trainable Params	1.2M (1.1%)	110M (100%)
Training Time	~2 minutes	~14 minutes

GPU Memory	4 GB	16 GB
Accuracy	82%	89%
Recall	76%	84%

I went with full fine-tuning for the final model since I had a decent GPU and recall was the priority. But frozen fine-tuning is a valid default when you're working on a limited compute budget or your dataset is small (under ~5,000 examples) overfitting becomes a real risk with full fine-tuning there.

4.5 Positional Encoding

Transformers process all tokens in parallel, so they have no inherent sense of word order. ClinicalBERT uses sinusoidal positional encodings added to token embeddings to inject order information. This matters for clinical text because 'antibiotics discontinued, patient improving' and 'patient improving after antibiotics' describe very different clinical situations — same words, different order.

4.6 Parallelization Advantage

RNNs process sequences one step at a time, so inference time scales with sequence length $O(T)$. Transformers process all tokens at once, so per-layer time is effectively $O(1)$ for parallelism. For a 500-bed hospital generating around 50 notes per hour, a single V100 can process everything in under 5 minutes. That's well within any reasonable clinical response window.

5. Deployment and Real-World Considerations

5.1 Recommended Architecture for Real ICU Use

Based on everything above, my recommended deployment is a hybrid ensemble. Not one model two, running on different timescales:

Real-time component (every 15 minutes): GRU on vital signs. It's fast (5ms per patient), unidirectional so it works in real-time, and catches rapid deterioration quickly.

Batch component (every 4 hours): ClinicalBERT on clinical notes. Higher accuracy, provides risk stratification and context for alerts.

Alert thresholds:

Alert Level	Condition	Action
Yellow	GRU > 0.7 OR ClinicalBERT > 0.6	Nurse review
Red	GRU > 0.8 AND ClinicalBERT > 0.7	Physician — immediate
Green	Both < 0.5	Routine monitoring

5.2 Ethical Issues I Think About

Bias is probably the biggest one. MIMIC-III, which my model is based on, has documented disparities in care quality across racial groups. A model trained on this data could potentially underperform for minority patients which in a high-stakes clinical setting is a serious problem. Any real deployment needs subgroup auditing, not just overall accuracy metrics.

Privacy is the other major concern. Clinical notes are full of protected health information. Deployment requires HIPAA-compliant infrastructure encryption at rest and in transit, access logs, and automatic de-identification before any data leaves the hospital.

And then there's accountability. If the model misses a sepsis case and a patient dies, who is responsible? The developer? The hospital? The doctor who got the alert and didn't act? These governance questions need clear answers before deployment. My position is that AI should always be decision support, never decision replacement. The physician has the final call, always.

5.3 Future Extension Multimodal with Chest X-rays

The natural next step would be adding imaging. A multimodal system would use a vision encoder (ResNet-50 or a Vision Transformer) to extract features from chest X-rays, ClinicalBERT for notes, and a cross-attention fusion layer to align visual findings with textual descriptions. For example: 'bilateral opacities' in a radiology report correlating with actual infiltrates visible in the X-ray. This mirrors how radiologists actually work reading the report and looking at the image together. For pneumonia-related sepsis, where imaging often shows findings before blood results confirm them, this could be a real clinical advantage.

Conclusion

Looking back at the progression DNN to LSTM/GRU to ClinicalBERT each model addressed a real limitation of the one before. The DNN gave me a baseline and forced me to think carefully about class imbalance and regularization. The recurrent models brought in temporal awareness that the problem genuinely requires. ClinicalBERT unlocked the clinical notes, which turned out to add meaningful signal on top of the vitals. The ensemble puts it all together.

The final numbers 89% accuracy, 84% recall are encouraging, but I don't think numbers are the main takeaway. The more important point is that this system is explainable and designed with clinical reasoning in mind. The attention heatmaps, the alert thresholds, the real-time vs batch split these design choices came from thinking about how doctors and nurses actually work, not just from optimizing a loss function.

AI isn't going to replace doctors. But it can be the system that watches every patient, every minute, and catches what a tired human might miss at 3am. That seems worth building.